Tutorial: the pipes kit

This walks through building the pipes kit from nothing and solving it, using only the LamdaFunction sidebar. It is the phase 1 gate: if you can follow it without asking, the extension does its job. About 20 minutes.

What you make: six pipe pieces that connect only where a pipe meets a pipe, fill a box with them, and get a closed network with no open ends on the outside.

1. Install

You need Blender 5.2 or newer and one extension zip for your platform. Get the zip in one of two ways.

Download it. Go to https://github.com/arlav/lamdafunction/releases, open the latest release and download the zip for your machine: `lamdafunction-<version>-windows_x64.zip`, `-macos_arm64.zip` (Apple Silicon), `-macos_x64.zip` (Intel Mac) or `-linux_x64.zip`. If your teacher sent you a zip, that is the same file.

Or build it. Clone https://github.com/arlav/lamdafunction, install Blender, and run from the repository root:

```
python scripts/build_extension.py
```

It finds your Blender, vendors the Python solver and writes the four zips into `dist/`. To include the native solver as well, build its wheel first (`maturin build --release -m core/lamda-py/Cargo.toml --out addon/wheels`, needs Rust) and list it in `addon/blender_manifest.toml`.

Then install the zip:

1. In Blender: Edit → Preferences → Get Extensions. Click the drop-down arrow at the top right of that page and choose **Install from Disk**. Pick the zip.
2. Still in Preferences, open **Add-ons**, search for `lamda` and make sure **LamdaFunction** is ticked.
3. In the 3D viewport press `N` and open the **LamdaFunction** tab in the sidebar. If you do not see it, the sidebar has more tabs than fit: scroll the tab strip or drag the sidebar wider.

Windows and Linux zips currently solve with the pure Python solver; the macOS Apple Silicon zip also carries the native one. For this tutorial there is no difference.

2. The library

1. In **Library**, click **New Library**. A collection called "LamdaFunction Library" appears in the outliner. Every child collection of it is a module.
2. Set **Cell size** to 1. Every part is modelled to fit a 1 × 1 × 1 cage centred on its own origin.

3. Click **Add Module** six times, naming them `straight`, `elbow`, `tee`, `cross`, `cap`, `empty`. Each gets a cube; replace the cube with pipe geometry as you like, or keep the cube and just set the faces. The order in the outliner is the library order: do not rename or reorder after solving unless you re-solve.

Model each piece around the origin of its collection (or set the collection's instance offset to the piece's centre). The connector arrows in the viewport show you the cage.

3. Faces and connectors

Select an object of a module. **Faces and Connectors** lists its six faces: +X, –X, +Y, –Y, +Z, –Z. Type a connector name on each face:

Module	"t" on	"0" on
straight	+X, –X	the other four
elbow	+X, +Y	the other four
tee	+X, –X, +Y	the other three
cross	+X, –X, +Y, –Y	+Z, –Z
cap	+X	the other five
empty	none	all six

Leave every **offset** at 0: pipes are round, so the arrow direction does not matter. Leave the terminator toggles off.

In **Library**, set **Rotation** to **Full** for every module. The compiler generates the rotations: straight has 3 distinct ones, elbow 12, tee 12, cross 3, cap 6, empty 1, 37 in all. You will see that number in the audit.

4. Rules

In **Rules**:

1. Turn **Indifference** off. Every face carries a name, so nothing is uncovered.
2. Add two connector pairs: `t` with `t`, and `0` with `0`. Leave all four twist toggles on (0°, 90°, 180°, 270°): a round pipe fits at any twist.

That is the whole rule set: a pipe end meets a pipe end, a blank face meets a blank face, and nothing else.

5. Envelope

In **Envelope**, click **Box Envelope** and choose 4 × 3 × 2. A point cloud of 24 vertices appears, one per slot; it carries the `wfc_cell` attribute. Any mesh works as an envelope: its vertices are the slots, so a Geometry Nodes modifier can generate one.

Keep **Closed boundary** on with connector `0`: every face on the outside of the box must be blank, so no pipe ends in mid-air.

6. Solve

In **Solve**, leave seed 1, 4 attempts, 100 backtracks. Click **Compile** first: **Results and Audit** shows "37 variants from 6 modules" and no errors. Then click **Solve**. The status bar counts slots while it runs; Esc cancels. With this kit it takes a few milliseconds.

The result is a new object, "LamdaFunction Assembly", with one vertex per slot and the `wfc_*` attributes on it (module, rotation, state, step, …). Change the seed and solve again for another network.

7. See it

- **Materialise** adds a Geometry Nodes modifier to the assembly that instances your modules on the solved slots. Its **Realize** input turns instances into real geometry for export.
- **Inspect** adds a modifier that shows each slot as a cube: green solved, red contradiction, grey undecided. Turn on its **Playback** input and scrub the timeline to watch the collapse order.

What to try next

- Give `cap` weight 0.2 and `straight` weight 3 in **Library**: fewer dead ends, longer runs.
- Set **Closed boundary** off: pipes may now leave the box.
- Paint a float attribute `wfc_w:cap` on the envelope with 0 in the middle rows: no caps there.
- Add a rule `cap:+X -> cap:-X` as **Disallowed**: two caps never face each other.

If something cannot be solved, the audit says which slot empties first; **Select Culprit** moves the 3D cursor there.